



ITESO

**Universidad Jesuita
de Guadalajara**

David Castañeda Quintero

Materia: Servicios seguros de internet

M6 - Proyecto Final.

M6 - Proyecto Final.....	1
Introducción.....	3
Server 1.....	4
Instalar Apache.....	4
Bind9 y dominios.....	10
Server 1 Hardening de seguridad.....	13
Server 2.....	15
Instalar SNMP.....	15
Descargar Zabbix.....	16
Instalación de Grafana y plugin de Zabbix.....	21
Script de backups.....	31
Hardening Server 2.....	33
Conclusiones.....	35
Referencias.....	36

Introducción

Este proyecto tiene como objetivo la configuración y despliegue de dos servidores Linux plenamente funcionales, cada uno con servicios específicos que colaboran en conjunto para lograr un entorno robusto, seguro y automatizado. El primer servidor estará enfocado en ofrecer servicios web mediante un stack LAMP con HTTPS utilizando certificados autofirmados, así como resolución DNS y actuar como servidor central de logs mediante rsyslog. Por su parte, el segundo servidor será responsable de la supervisión y monitoreo continuo del sistema, utilizando SNMP, Zabbix y Grafana, además de ejecutar tareas programadas para respaldos y gestión de logs.

Ambos servidores deberán cumplir con políticas específicas de seguridad, incluyendo la configuración de firewalld, la aplicación de políticas SELinux en modo Enforcing, y la segmentación de usuarios en grupos con distintos niveles de privilegios. Asimismo, se aplicarán prácticas de hardening como la restricción del acceso SSH al usuario root, el uso exclusivo de autenticación por llaves, la modificación del puerto por defecto y la prohibición de login por contraseña.

La integración de estos elementos permitirá no solo un entorno funcional, sino también una infraestructura alineada con buenas prácticas de administración en sistemas Linux.

Server 1

Instalar Apache

```
root@172-15-0-10:/home/adrian# dnf install -y httpd
Updating and loading repositories:
Fedora 42 openh264 (From Cisco) - x86_64      100% | 1.0 KiB/s | 6.0 KiB | 00m06s
Fedora 42 - x86_64 - Updates                  100% | 1.1 MiB/s | 5.2 MiB | 00m05s
Fedora 42 - x86_64                            100% | 1.6 MiB/s | 35.4 MiB | 00m22s
Repositories loaded.
Package "httpd-2.4.62-6.fc42.x86_64" is already installed.
Nothing to do.
```

```
root@172-15-0-10:/home/adrian# systemctl start httpd.service
root@172-15-0-10:/home/adrian# systemctl status httpd.service
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf, 50-keep-warm.conf
   Active: active (running) since Mon 2025-04-28 12:31:42 CST; 4min 10s ago
   Invocation: ae1ee451cc034f0f8dbec85ce9be2f8a
   Docs: man:httpd.service(8)
  Main PID: 2126 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 4583)
   Memory: 19.5M (peak: 20.1M)
     CPU: 1.205s
    CGroup: /system.slice/httpd.service
            └─2126 /usr/sbin/httpd -DFOREGROUND
              └─2127 /usr/sbin/httpd -DFOREGROUND
                └─2129 /usr/sbin/httpd -DFOREGROUND
                  └─2130 /usr/sbin/httpd -DFOREGROUND
                    └─2131 /usr/sbin/httpd -DFOREGROUND
```

Arrancamos el servicio

Instalación y habilitación de Apache

```
root@172-15-0-10:/home/adrian# sudo dnf install mysql-server
Updating and loading repositories:
Repositories loaded.
Package Arch Version Repository Size
Installing:
mysql-server x86_64 8.0.41-2.fc42 updates 108.2 MiB
Installing dependencies:
mecab x86_64 0.996-9.fc42 fedora 1.2 MiB
mysql x86_64 8.0.41-2.fc42 updates 58.3 MiB
mysql-common noarch 8.0.41-2.fc42 updates 442.9 KiB
mysql-errmsg noarch 8.0.41-2.fc42 updates 9.9 MiB
mysql-selinux noarch 1.0.13-2.fc42 fedora 49.8 KiB
protobuf-lite x86_64 3.19.6-11.fc42 fedora 861.0 KiB

Transaction Summary:
Installing: 7 packages

Total size of inbound packages is 21 MiB. Need to download 21 MiB.
After this operation, 179 MiB extra will be used (install 179 MiB, remove 0 B).
Is this ok [y/N]: y
```

Instalación mysql

```

root@172-15-0-10:/home/adrian# dnf -y install php php-mysqld php-cli
Updating and loading repositories:
Repositories loaded.
Package Arch Version Repository Size
Installing:
php x86_64 8.4.6-1.fc42 updates 0.0 B
php-cli x86_64 8.4.6-1.fc42 updates 12.2 MiB
php-mysqld x86_64 8.4.6-1.fc42 updates 381.9 KiB
Installing dependencies:
nginxfilesystem noarch 2:1.26.3-1.fc42 fedora 141.0 B
php-common x86_64 8.4.6-1.fc42 updates 9.5 MiB
php-pdo x86_64 8.4.6-1.fc42 updates 238.2 KiB
Installing weak dependencies:
php-fpm x86_64 8.4.6-1.fc42 updates 6.2 MiB
php-mbstring x86_64 8.4.6-1.fc42 updates 1.2 MiB
php-opcache x86_64 8.4.6-1.fc42 updates 1.2 MiB
php-sodium x86_64 8.4.6-1.fc42 updates 109.5 KiB
php-xml x86_64 8.4.6-1.fc42 updates 3.5 MiB

Transaction Summary:
Installing: 11 packages

```

Instalación de php

```

root@172-15-0-10:/home/adrian# dnf -y install php php-mysqld php-cli
Updating and loading repositories:
Repositories loaded.
Package Arch Version Repository Size
Installing:
php x86_64 8.4.6-1.fc42 updates 0.0 B
php-cli x86_64 8.4.6-1.fc42 updates 12.2 MiB
php-mysqld x86_64 8.4.6-1.fc42 updates 381.9 KiB
Installing dependencies:
nginxfilesystem noarch 2:1.26.3-1.fc42 fedora 141.0 B
php-common x86_64 8.4.6-1.fc42 updates 9.5 MiB
php-pdo x86_64 8.4.6-1.fc42 updates 238.2 KiB
Installing weak dependencies:
php-fpm x86_64 8.4.6-1.fc42 updates 6.2 MiB
php-mbstring x86_64 8.4.6-1.fc42 updates 1.2 MiB
php-opcache x86_64 8.4.6-1.fc42 updates 1.2 MiB
php-sodium x86_64 8.4.6-1.fc42 updates 109.5 KiB
php-xml x86_64 8.4.6-1.fc42 updates 3.5 MiB

Transaction Summary:
Installing: 11 packages

```

Habilitamos SQL

```

root@172-15-0-10:/home/adrian# systemctl enable mysqld.service
Created symlink '/etc/systemd/system/multi-user.target.wants/mysqld.service' → '/usr/lib/systemd/system/mysqld.service'.
root@172-15-0-10:/home/adrian#
root@172-15-0-10:/home/adrian#
root@172-15-0-10:/home/adrian# systemctl start mysqld.service
root@172-15-0-10:/home/adrian# mysql -u root -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.41 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

```

Iniciamos sesión como root, y arrancamos el servicio

```

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.01 sec)

mysql> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.02 sec)

```

Se accede correctamente a nuestra bdd llamada wordpress_db

```
mysql> GRANT ALL ON wordpress_db.* TO 'wordpress_user'@'localhost' IDENTIFIED BY 'Pa$$w0rd';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
for the right syntax to use near 'IDENTIFIED BY 'Pa$$w0rd'' at line 1
mysql> CREATE USER 'wordpress_user'@'localhost' IDENTIFIED BY 'Passw0rd';
Query OK, 0 rows affected (0.08 sec)

mysql> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wordpress_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> exit;
Bye
root@172-15-0-10:/home/adrian#
```

Concebimos permisos de ejecución a nuestro usuario designado. Finalmente hacemos flush privileges para actualizar la tabla de datos y los cambios sean aplicados.

Instalación de wordpress

```
root@172-15-0-10:/home/adrian# wget https://wordpress.org/latest.tar.gz
Saving 'latest.tar.gz'
HTTP response 200 [https://wordpress.org/latest.tar.gz]
latest.tar.gz 100% [=====] 25.67M 4.20MB/s
[Files: 1 Bytes: 25.67M [2.60MB/s] Redirects: 0 Todo: 0 Errors: 0 ]
root@172-15-0-10:/home/adrian#
```

```
root@172-15-0-10:/home/adrian# tar -xvf latest.tar.gz
wordpress/
wordpress/index.php
wordpress/license.txt
wordpress/readme.html
wordpress/wp-activate.php
wordpress/wp-admin/
wordpress/wp-admin/about.php
wordpress/wp-admin/admin-ajax.php
wordpress/wp-admin/admin-footer.php
wordpress/wp-admin/admin-functions.php
```

desempaquetado de wordpress

```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress_db' );

/** Database username */
define( 'DB_USER', 'wordpress_user' );

/** Database password */
define( 'DB_PASSWORD', 'Passw0rd' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

Editamos el archivo de configuración de Wordpress, en este definimos un usuario, contraseña y un nombre de BDD

```

root@172-15-0-10:/home/adrian# cp -R wordpress /var/www/html/
root@172-15-0-10:/home/adrian# chown -R apache:apache /var/www/html/wordpress/
root@172-15-0-10:/home/adrian# chcon -t httpd_sys_rw_content_t /var/www/html/wordpress/ -R
root@172-15-0-10:/home/adrian# chmod -Rf 775 /var/www/html

```

Principalmente cambiamos el contexto de tipo SELinux para permitir accesos al servidor web. Además permisos absolutos a propietario, grupo y a otros lectura con ejecución.

```

root@172-15-0-10:/home/adrian# cat /etc/httpd/conf.d/wordpress.conf
<VirtualHost *:80>
ServerAdmin admin@localhost
DocumentRoot /var/www/html/wordpress

<Directory "/var/www/html/wordpress">
Options Indexes FollowSymLinks
AllowOverride all
Require all granted
</Directory>

ErrorLog /var/log/httpd/wordpress_error.log
CustomLog /var/log/httpd/wordpress_access.log common
</VirtualHost>
root@172-15-0-10:/home/adrian# |

```

editamos archivo de config de wordpress, configuramos rutas y puerto

```

root@172-15-0-10:/home/adrian# systemctl restart httpd.service
root@172-15-0-10:/home/adrian# semanage fcontext -a -t httpd_sys_rw_content_t "/var/www/html/wordpress(/.*)?"

root@172-15-0-10:/home/adrian#
root@172-15-0-10:/home/adrian# restorecon -Rv /var/www/html/wordpress
root@172-15-0-10:/home/adrian# systemctl enable httpd
Created symlink '/etc/systemd/system/multi-user.target.wants/httpd.service' → '/usr/lib/systemd/system/httpd.service'.
root@172-15-0-10:/home/adrian# systemctl enable mysqld.service
root@172-15-0-10:/home/adrian# |

```

Generación certificados

```

root@172-15-0-10:/home/adrian# openssl genrsa -out /etc/pki/tls/private/mykey.key 2048
root@172-15-0-10:/home/adrian# openssl req -new -key /etc/pki/tls/private/mykey.key -out /etc/pki/tls/certs/myselfsigned
d.csr -subj "/C=MX/ST=Jalisco/L=Tlaquepaque/O=ITESO AC/OU=DESI/CN=lamp/emailAddress=adrian.galindovilla@iteso.mx"

```

Este comando genera una clave privada RSA de 2048 bits y la guarda en el directorio estándar de claves privadas. Esta clave es esencial para el cifrado y será utilizada para firmar el certificado.

```

root@172-15-0-10:/home/adrian# openssl x509 -req -days 365 -in /etc/pki/tls/certs/myselfsigned.csr -signkey /etc/pki/tls
/private/mykey.key -out /etc/pki/tls/certs/myselfsigned.crt
Certificate request self-signature ok
subject=C=MX, ST=Jalisco, L=Tlaquepaque, O=ITESO AC, OU=DESI, CN=lamp, emailAddress=adrian.galindovilla@iteso.mx

```

Con este comando se genera una Certificate Signing Request (CSR), que contiene información del servidor (como nombre común, organización, etc.) y la clave pública. Aunque normalmente esta solicitud se enviaría a una autoridad certificadora (CA), en este caso se usará para crear un certificado autofirmado.

```
root@172-15-0-10:/home/adrian# cat /etc/httpd/conf.d/wordpress.conf
<VirtualHost *:443>
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html/wordpress

    ServerName 172.15.0.10

    SSLEngine on
    SSLCertificateFile /etc/pki/tls/certs/myselfsigned.crt
    SSLCertificateKey /etc/pki/tls/private/mykey.key

<Directory /var/www/html/wordpress>
    AllowOverride All
    Require all granted
</Directory>

ErrorLog /var/log/httpd/wordpress-ssl-error.log
CustomLog /var/log/httpd/wordpress-ssl-access.log combined
</VirtualHost>
root@172-15-0-10:/home/adrian#
```

Especificamos la ruta del certificado, llave y puertos (443), en el archivo de config de wordpress

Success!

WordPress has been installed. Thank you, and enjoy!

Username admin

Password *Your chosen password.*

[Log In](#)

una vez accedemos a wordpress, creamos usuario y contraseña.

172.15.0.10/wp-admin/

MyWordpress

Edit Site

0

New

Howdy, admin

Screen Options

Help

Dashboard

Home

Updates

Posts

Media

Pages

Comments

Appearance

Plugins

Users

Tools

Settings

Collapse Menu

Dashboard

Welcome to WordPress!

[Learn more about the 6.8 version.](#)

Author rich content with blocks and patterns

Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.
[Add a new page](#)

Customize your entire site with block themes

Design everything on your site — from the header down to the footer, all using blocks and patterns.
[Open site editor](#)

Switch up your site's look & feel with Styles

Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?
[Edit styles](#)

Site Health Status

No information yet...

Site health checks will automatically run periodically to gather information about your site. You can also [visit the Site Health screen](#) to gather information about your site now.

Quick Draft

Title

Content

Regular

MyWordpress

Sample Page

Blog

Hello world!

Welcome to WordPress. This is your first post. Edit or delete it, then start writing!

April 28, 2025

Bind9 y dominios

Primero instalamos Bind9

```
root@172-15-0-10:/home/adrian# dnf install bind bind-utils -y
Updating and loading repositories:
Repositories loaded.
Package "bind-utils-32:9.18.33-1.fc42.x86_64" is already installed.

Package Arch Version Repository Size
Upgrading:
bind-libs x86_64 32:9.18.35-2.fc42 updates 3.6 MiB
replacing bind-libs x86_64 32:9.18.33-1.fc42 624cbc92582d4ecfae4c58749abde4f 3.6 MiB
bind-utils x86_64 32:9.18.35-2.fc42 updates 665.1 KiB
replacing bind-utils x86_64 32:9.18.33-1.fc42 624cbc92582d4ecfae4c58749abde4f 665.2 KiB
Installing:
bind x86_64 32:9.18.35-2.fc42 updates 877.1 KiB
Installing weak dependencies:
bind-dnssec-utils x86_64 32:9.18.35-2.fc42 updates 440.0 KiB

Transaction Summary:
Installing: 2 packages
Upgrading: 2 packages
Replacing: 2 packages

Total size of inbound packages is 2 MiB. Need to download 2 MiB.
After this operation, 1 MiB extra will be used (install 6 MiB, remove 4 MiB).
```

Al querer resolver direcciones A a dominios, agregamos en el archivo de configuración de Bind9, dominio y direcciones.

```
$TTL 3600
@      IN      SOA      ns1.proyecto. admin.proyecto. (
        2025040301 ; Serial (YYYYMMDDnn format)
        3600      ; Refresh
        1800      ; Retry
        604800    ; Expire
        86400     ; Minimum TTL
)

@      IN      NS       ns1.blog.
ns1    IN      A        172.15.0.10
primario IN      IN      A        172.15.0.10
blog   IN      A        172.15.0.10
secundario IN      A      172.15.0.20
~
```

```
zone "proyecto" IN {
    type master;
    file "/var/named/blog";
};
```

creamos la zona, en este caso "proyecto"

```
options {
    listen-on port 53 { 127.0.0.1; 172.15.0.10; };
    listen-on-v6 port 53 { ::1; };
    directory      "/var/named";
    dump-file       "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    secroots-file   "/var/named/data/named.secroots";
    recursing-file  "/var/named/data/named.recursing";
    allow-query     { localhost; 172.15.0.0/24; };
}
```

Configuración básica de BIND que define la carpeta base, archivos de estadísticas, zonas y caché. Especifica interfaces que escuchan en IPv4/IPv6, permisos de consulta (solo red local) y rutas para estadísticas, DNSSEC y recursión.

```
-rw-r-----. 1 root named 168 Mar 25 18:00 named.loopback
-rw-r--r--. 1 root named 433 Apr 28 15:11 proyecto
```

En los archivos de configuración se asigna a named como usuario

```
> > primario.proyecto
Server:      localhost
Address:     ::1#53

Name:   primario.proyecto
Address: 172.15.0.10
> blog.proyecto
Server:      localhost
Address:     ::1#53

Name:   blog.proyecto
Address: 172.15.0.10
> secundario.proyecto
Server:      localhost
Address:     ::1#53

Name:   secundario.proyecto
Address: 172.15.0.20
> |
```

Hacemos un `nslookup` para consultar servidores DNS, y obtener información sobre nuestros nombres de dominios así como de sus direcciones ip, con esto corroborando la efectiva configuración

Rsyslog

```
root@172-15-0-10:/home/adrian# dnf install rsyslog
Updating and loading repositories:
Repositories loaded.
Package "rsyslog-8.2412.0-3.fc42.x86_64" is already installed.
```

```
Nothing to do.
```

```
root@172-15-0-10:/home/adrian#
```

```
Nothing to do.
```

```
root@172-15-0-10:/home/adrian# systemctl enable rsyslog
root@172-15-0-10:/home/adrian# systemctl start rsyslog
root@172-15-0-10:/home/adrian# systemctl status rsyslog
● rsyslog.service - System Logging Service
   Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf, 50-keep-warm.conf
   Active: active (running) since Mon 2025-04-28 14:47:42 CST; 41min ago
```

Configuración Server 1

```
# For TCP reception (port 514/tcp):
module(load="imtcp")
input(type="imtcp" port="514")
|
"/etc/rsyslog.conf" 87L, 3494B
```

Y evidentemente también tenemos que abrir el puerto 514 en Firewalld (explicado más adelante).

Configuración Server 2 de rsyslog

Dentro del archivo `/etc/rsyslog.d/remote.conf` escribimos la siguiente línea:

```
*.* @172.15.0.10:514
```

El primer asterisco representa que todos los servicios van a enviarse (kernel logs, daemons, cron...), mientras que el segundo significa el nivel de severidad (todos). O sea, "Enviar

todos los logs de todos los servicios independientemente del nivel de severidad". Después, la IP del servidor receptor (Server 1) y el puerto.

En Server 2 podemos hacer esta prueba

```
root@172-15-0-20:/home/adrian# logger "Here be dragons!"
root@172-15-0-20:/home/adrian# systemctl start firewalld.service
root@172-15-0-20:/home/adrian# logger "Here be more dragons!"
```

Y en Servidor 1 podemos ver el archivo /var/log/messages y nos daremos cuenta que ahí nos llegan los avisos relacionados a Server 2.

```
May  1 17:47:33 172-15-0-20 root[1910]: This is a test
May  1 17:48:18 172-15-0-20 root[1919]: This is a test
May  1 17:50:18 172-15-0-20 root[1923]: Here be dragons!
May  1 17:50:26 172-15-0-20 audit: BPF prog-id=62 op=LOAD
May  1 17:50:26 172-15-0-20 systemd[1]: Starting firewalld.service - firewalld - dynamic firewall daemon...
May  1 17:50:27 172-15-0-20 systemd[1]: Started firewalld.service - firewalld - dynamic firewall daemon.
May  1 17:50:27 172-15-0-20 audit[1]: SERVICE_START pid=1 uid=0 auid=4294967295 ses=4294967295 subj=system_u
=? terminal=? res=success'
May  1 17:50:27 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld_probe:11 family=1 entries=1 op=nft_re
May  1 17:50:27 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld_probe:12 family=1 entries=1 op=nft_ur
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld:13 family=1 entries=1 op=nft_register
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld:13 family=1 entries=2 op=nft_unregist
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld:14 family=1 entries=232 op=nft_regist
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld_policy_drop:15 family=1 entries=2 op=
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld:16 family=1 entries=26 op=nft_registe
May  1 17:50:28 172-15-0-20 audit[1940]: NETFILTER_CFG table=firewalld:17 family=1 entries=34 op=nft_registe
May  1 17:50:41 172-15-0-20 root[1967]: Here be more dragons!
```

Podemos apreciar los mensajes que enviamos, además de las notificaciones de que el servicio de Firewalld se inició.

Server 1 Hardening de seguridad

SSH

Dentro del archivo de configuración /etc/ssh/sshd_config editamos las siguientes líneas

denegar permisos para loggear como root
PermitRootLogin no AA

permitir login mediante key authentication
PubKeyAuthentication yes

Esto es para que SELinux permita cambiar el puerto
sudo semanage port -a -t ssh_port_t -p tcp 2222

Firewalld

```
root@172-15-0-10:/home/adrian# firewall-cmd --zone=work --change-interface=ens224 --permanent
The interface is under control of NetworkManager, setting zone to 'work'.
success
root@172-15-0-10:/home/adrian# firewall-cmd --zone=work --add-port=2222/tcp
success
root@172-15-0-10:/home/adrian# firewall-cmd --zone=work --add-port=2222/tcp --permanent
success
root@172-15-0-10:/home/adrian# firewall-cmd --zone=work --add-service=dns --permanent
success
```

Asigna la interfaz ens224 a la zona work del firewall. Abre el puerto TCP 2222 (temporal y permanente) y permite el servicio DNS de forma permanente en esa zona.

```
root@172-15-0-10:/home/adrian# firewall-cmd --zone=work --list-all
work (active)
target: default
ingress-priority: 0
egress-priority: 0
icmp-block-inversion: no
interfaces: ens224
sources:
services: dhcpv6-client dns mdns ssh
ports: 2222/tcp 514/tcp
protocols:
forward: yes
masquerade: no
forward-ports:
source-ports:
icmp-blocks:
rich rules:
root@172-15-0-10:/home/adrian# |
```

Zona work

```
root@172-15-0-10:/home/adrian# firewall-cmd --zone=public --list-all
public (active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: dhcpv6-client https mdns
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
root@172-15-0-10:/home/adrian# |
```

Zona public

Sobre la configuración de SELinux, es la misma que se le aplicará a Server 2, así que estará mostrada en dicho apartado.

Server 2

Instalar SNMP

```
sudo dnf install net-snmp net-snmp-utils
```

net-snmp instala lo necesario para que el protocolo SNMP funcione. Net-snmp instala las herramientas para poder interactuar con el protocolo y el demonio que se va a encargar de administrarlo.

Después, editamos el archivo de configuración /etc/snmp/snmpd.conf

```

# Red en la que se va a comunicar
com2sec local 127.0.0.1/32 MyGroup
com2sec mynetwork 172.15.0.0/24 MyGroup

# Grupos
group MyROGroup v2c local
group MyROGroup v2c mynetwork

#Vistas
view all included .1 80

#Permisos
access MyROGroup "" v2c noauth exact all none none

```

Es necesario que habilitemos SNMP

```
systemctl enable snmpd
```

```
systemctl start snmpd
```

Descargar Zabbix

```
vim /etc/yum.repos.d/epel.repo
```

abre o crea el archivo del repositorio epel para editarlo

```
rpm -Uvh
```

instala el repositorio oficial de zabbix

```
dnf clean all
```

limpia la caché de dnf para evitar conflictos

dnf install instala zabbix, apache, mariadb y dependencias ignorando verificación gpg y permitiendo reemplazos si hay conflictos

<https://repo.zabbix.com/zabbix/7.2/release/rhel/9/noarch/zabbix-release-latest-7.2.el9.noarch.rpm>

```
dnf install -y zabbix-server-mysql zabbix-web-mysql zabbix-apache-conf
zabbix-sql-scripts zabbix-agent mariadb-server --nogpgcheck --allowerasing
```

```
systemctl enable --now mariadb
```

mysql_secure_installation "Aquí cambiamos el password del usuario root de la DB. En nuestro caso, lo cambiamos por Pa\$\$w0rd"

`mysql -u root -p`

```
root@172-15-0-20:/home/adrian# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE zabbix character set utf8mb4 collate utf8mb4_bin;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'Pa$$w0rd';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
```

Se descomprime e importa el esquema de la base de datos de Zabbix a MySQL, usando el usuario zabbix.

```
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
```

Se configura un bloque server de NGINX para servir la interfaz web de Zabbix desde el puerto 8080.

```
vim /etc/nginx/conf.d/zabbix.conf
```

```
server {
    listen      8080;
    server_name Server2;

    root        /usr/share/zabbix/ui;

    index       index.php;
}
```

`vim /etc/zabbix_server.conf`

```
DBUser=zabbix

### Option: DBPassword
#       Database password.
#       Comment this line if no password is used.
#
# Mandatory: no
# Default:
DBPassword=Pa$$w0rd
```

Se establece el usuario y la contraseña que Zabbix usará para conectarse a la base de datos (DBUser=zabbix, DBPassword=Pa\$\$w0rd).

```
root@172-15-0-20:/home/adrian# systemctl restart zabbix-server zabbix-agent nginx php-fpm
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
```

Este error se dio porque no estaba instalado Nginx en server 2. Lo instalamos, volvimos a correr la instalación de Zabbix, y ahora sí se generaron los archivos de configuración correctos.

```
vim /etc/nginx/conf.d/zabbix.conf
systemctl restart zabbix-server zabbix-agent nginx php-fpm
systemctl enable zabbix-server zabbix-agent nginx php-fpm
```

Una vez corregido corre exitosamente

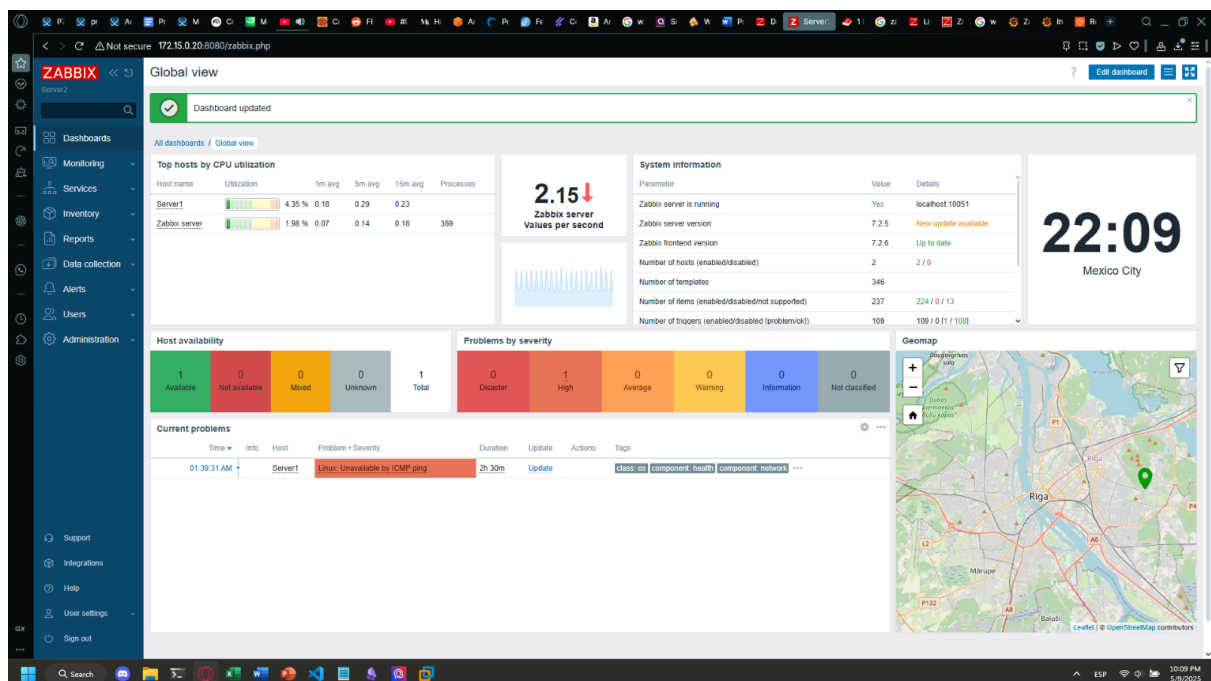
`systemctl restart zabbix-server`

```

● zabbix-server.service - Zabbix Server with MySQL
   Loaded: loaded (/usr/lib/systemd/system/zabbix-server.service; vendor preset: enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf, 50-keep-alive.conf
   Active: active (running) since Fri 2025-05-02 10:10:10 CEST; 1min ago
   Invocation: cdaecc4688cc4a32808a9de14ea58241
   Main PID: 3672 (zabbix_server)
   Tasks: 1 (limit: 4583)
   Memory: 2.6M (peak: 2.9M)
   CPU: 54ms
   CGroup: /system.slice/zabbix-server.service
           └─3672 /usr/sbin/zabbix_server -f

```

Ahora si abrimos el navegador en el puerto 8080 aparece el panel web de administración de Zabbix



Dentro del panel debemos agregar un nuevo host que será monitoreado por zabbix configurando su nombre ip grupo y agente

Host

Host

IPMI

Tags

Macros

Inventory

Encryption

Value mapping

* Host name

Server1

Visible name

Server1

Templates

Name

Linux by SNMP

type here to search

Select

Actions

Unlink Unlink and clear

* Host groups

Linux servers ✕

type here to search

Select

Interfaces

Type

SNMP

IP address

172.15.0.10

DNS name

Connect to

IP DNS

Port

161

Default

Remove

* SNMP version

SNMPv2

* SNMP community

MyGroup

Max repetition count

10

Use combined requests

Add

Description

Server1

Monitored by

Server Proxy Proxy group

Enabled

Update

Clone

Delete

Cancel

Como nombre, le asignamos server 1, y le agregamos su IP correspondiente (172.15.0.10). Además, usamos la plantilla de Linux SNMP, y agregamos el puerto 161 (el default para dicho protocolo). Algo muy importante es que en el campo de SNMP Community tenemos que poner el mismo nombre que pusimos en los archivos de configuración de SNMP (en este caso MyGroup).

Al inicio, en el dashboard de Zabbix aparecía que no estaba corriendo (en vez de “Yes” en verde decía “No” en rojo).

Parameter	Value	Details
Zabbix server is running	Yes	localhost:10051
Zabbix server version	7.2.5	New update available

Revisando los logs vimos que zabbix no podía acceder a la base de datos porque estaba leyendo otro archivo de configuración distinto al que habíamos editado así que corregimos eso editando el archivo correcto

```
using configuration file: /etc/zabbix_server.conf  
[Z3001] connection to database 'zabbix' failed: [1045] Access denied for user 'zabbix'@'local'
```

```
*****  
using configuration file: /etc/zabbix_server.conf  
[Z3001] connection to database 'zabbix' failed: [1045]
```

Una vez que editamos el archivo `/etc/zabbix_server.conf` y pusimos la contraseña correcta de la base de datos reiniciamos zabbix y el servidor ya aparecía corriendo

Instalación de Grafana y plugin de Zabbix

```
dnf install -y grafana
```

```
systemctl enable grafana-server
```

```
systemctl start grafana-server
```

Instalamos el plugin

```
grafana-cli plugins install alexanderzobnin-zabbix-app
```

Verificamos que exista dentro de la carpeta de plugins

```
root@172-15-0-20:/home/adrian# ls /var/lib/grafana/plugins/  
alexanderzobnin-zabbix-app  performancecopilot-pcp-app
```

Dentro del archivo `/etc/grafana/grafana.ini` buscamos la sección correspondiente a los plugins y agregamos el nombre del plugin que queremos usar en la lista de `allow loading unsigned plugins` esto permite que grafana cargue plugins que no están firmados oficialmente útil cuando usamos plugins personalizados o descargados manualmente.

```
allow_loading_unsigned_plugins = performancecopilot-valkey-datasource,performancecopilot-panel,alexanderzobnin-zabbix-app
```

Una vez que grafana estuvo instalado, podemos navegar al puerto 3000 para encontrar el portal de administración



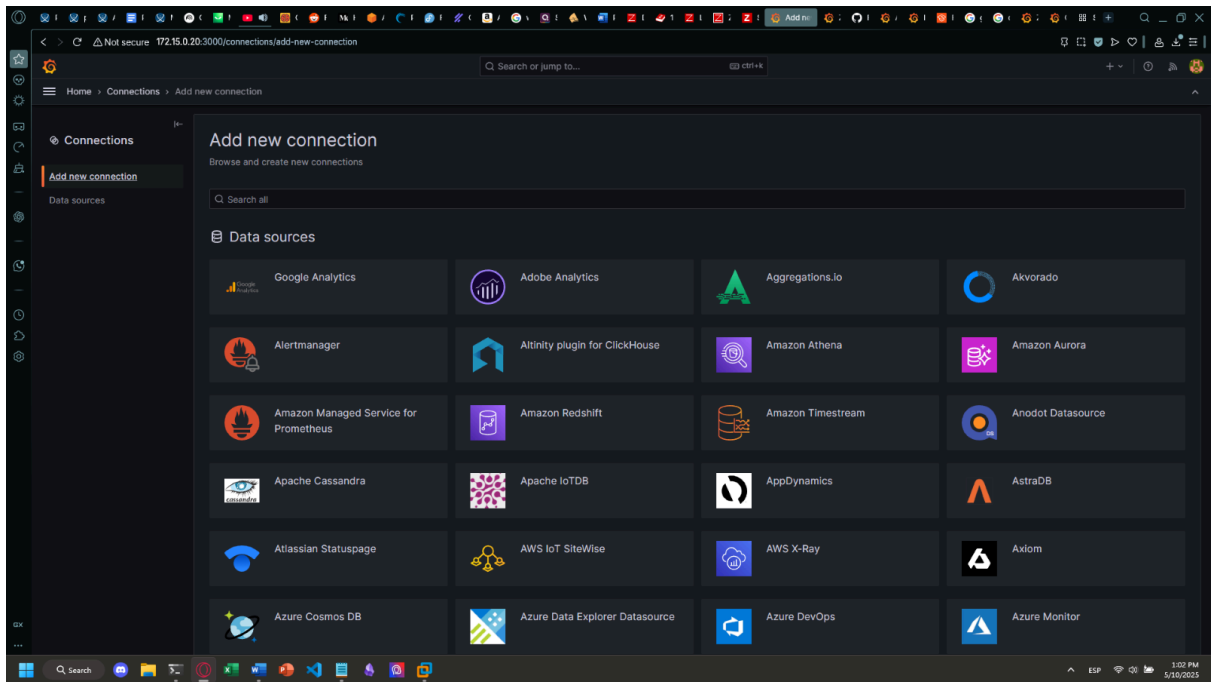
Welcome to Grafana

Email or username

Password

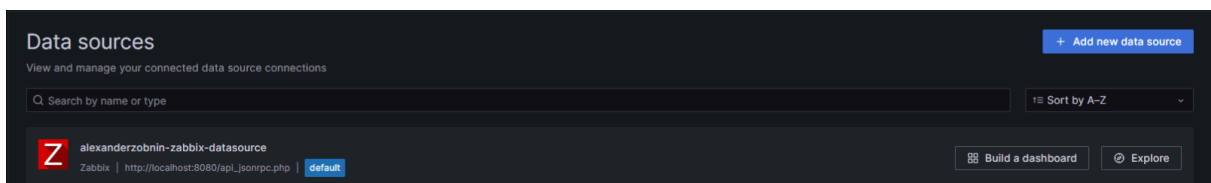
[Forgot your password?](#)

[Log in](#)



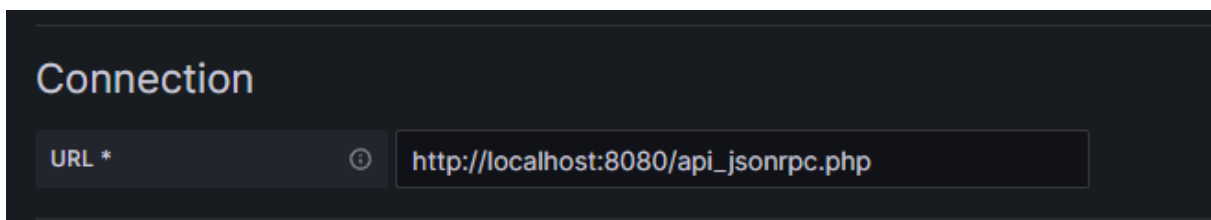
Agregar nueva fuente de datos en Grafana

En el panel de Grafana, navegamos a **Connections > Data sources** y seleccionamos el plugin de **Zabbix** de la lista de fuentes disponibles. Esto nos permite conectar Grafana con nuestro servidor Zabbix para visualizar métricas.



Verificación de la fuente de datos

Una vez agregado el plugin, aparece en la lista como “Zabbix data source”. Esto confirma que se ha reconocido correctamente y está lista para configurarse.

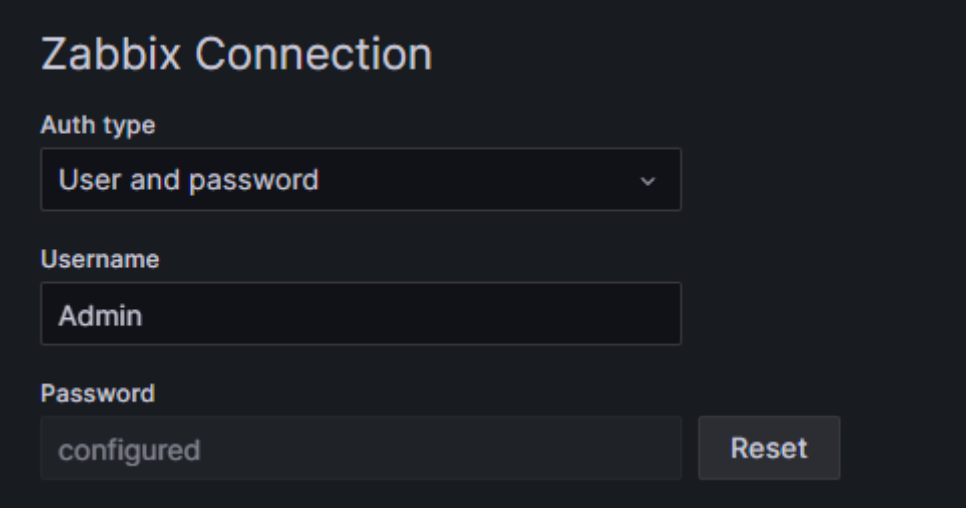


Configurar conexión a Zabbix

En el campo de URL, se especifica la dirección del API de Zabbix, que en este caso es:

http://localhost:8080/api_jsonrpc.php

Esta es la ruta estándar para que Grafana pueda comunicarse con Zabbix vía su API. Además, es necesario proveer las credenciales que se usan para comunicarse con el frontend de Zabbix (es decir, las mismas que se utilizan para acceder al panel de admin de Zabbix), que en este caso son Admin:zabbix



The image shows the 'Zabbix Connection' configuration form in Grafana. It has a dark theme. The title 'Zabbix Connection' is at the top. Below it, there are three sections: 'Auth type' with a dropdown menu set to 'User and password', 'Username' with a text input field containing 'Admin', and 'Password' with a text input field containing 'configured'. To the right of the password field is a 'Reset' button.

Field	Value
Auth type	User and password
Username	Admin
Password	configured

Dentro de Dashboards creamos uno y seleccionamos la métrica que queremos visualizar (en este caso, CPU)



The image shows the Grafana query editor interface. At the top, the data source is set to 'alexanderzobnin-zabbix-datas'. The query type is 'Metrics'. The group is 'Linux servers', the host is 'Server1', and the item tag is 'component: cpu'. The item is 'CPU utilization'. There are buttons for 'Add query' and 'Expression' at the bottom.

Field	Value
Query type	Metrics
Group	Linux servers
Host	Server1
Item tag	component: cpu
Item	CPU utilization



Creamos un nuevo dashboard desde cero y añadimos un panel. En la configuración del panel, seleccionamos **Zabbix** como fuente de datos y, en las opciones disponibles, elegimos la métrica que deseamos visualizar (por ejemplo: uso de CPU, espacio en disco, disponibilidad del host, etc.).

Grafana permite personalizar cada panel con gráficos, tablas o indicadores visuales, lo que nos da una representación clara y dinámica del estado de nuestros sistemas monitoreados.

Creando la regla de alerta

Edit rule

1. Enter alert rule name
Enter a name to identify your alert rule.

Name
CPU

2. Define query and alert condition
Define queries and/or expressions and then choose one of them as the alert rule condition. This is the threshold that

A **Z** alexanderzobnin-zabbix-data Options 6 hours, MD = 43200, Min. Interval =

Query type	Metrics		
Group	Linux servers	Host	Server1
Item tag	component: cpu	Item	CPU utilization
Functions	+		
> Options			

Definir el nombre de la alerta: Se asigna un nombre descriptivo a la regla de alerta, en este caso, "CPU", para identificarla fácilmente.

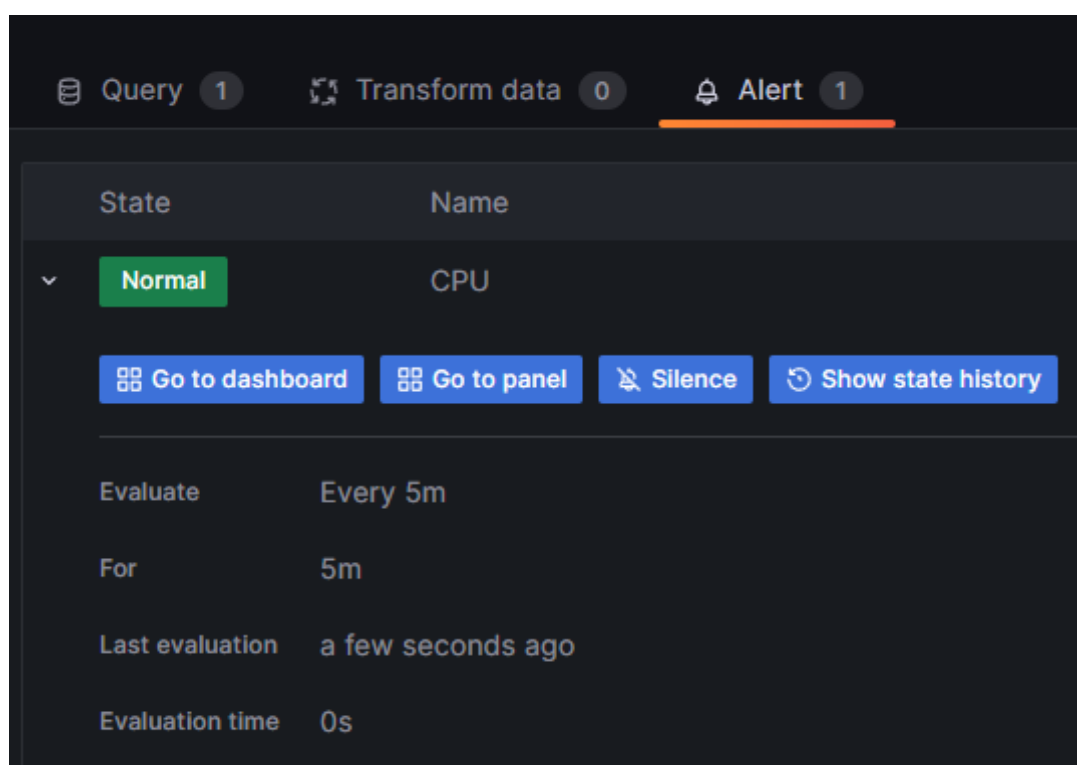
C Threshold ✓ Alert condition

Takes one or more time series returned from a query or an expression and checks if any of the series match the threshold condition.

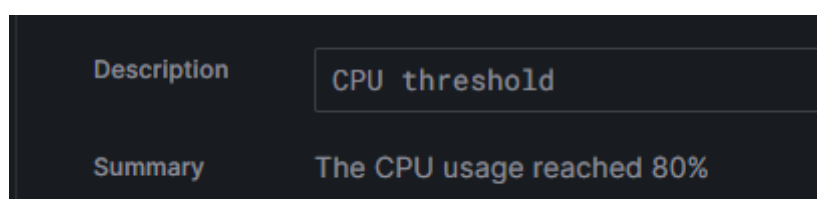
Input B

IS ABOVE 80

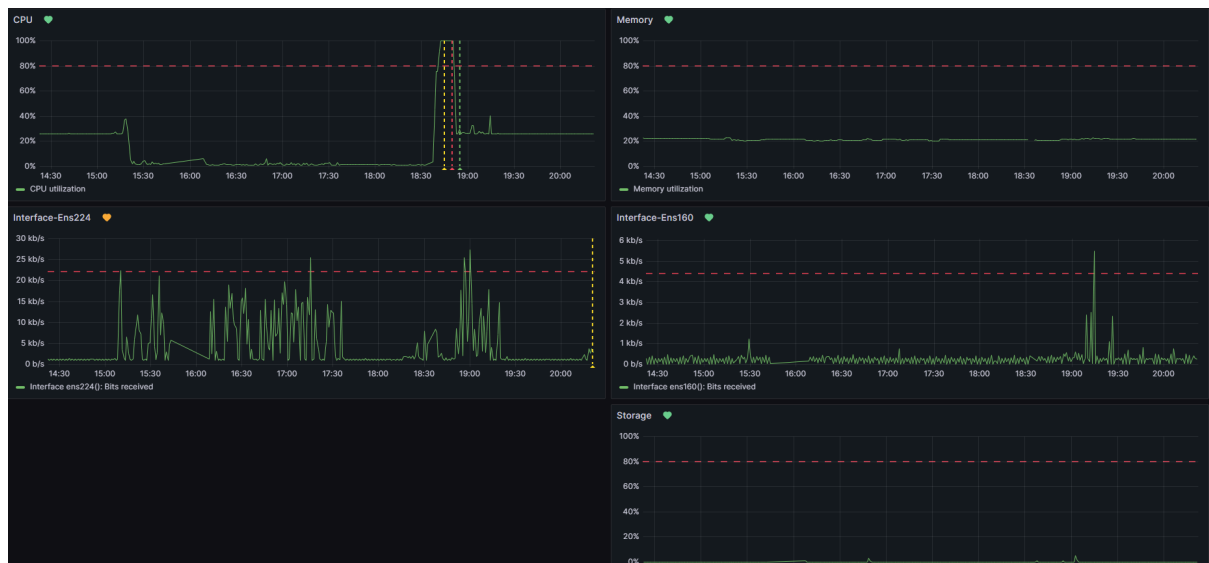
Definir la consulta y la condición de alerta: Se configura una consulta para monitorear métricas específicas del servidor, en este caso, la utilización de la CPU. La consulta está configurada para obtener los datos de "component: cpu" y utilizar el ítem "CPU utilization".



Finalmente, la configuración de la alerta se ajusta para ser "Normal", indicando que la alerta será activa y mostrará el historial de estado en el dashboard de Zabbix, proporcionando así un monitoreo eficiente y detallado del estado de la CPU.



Estos mismos pasos los vamos a realizar para los demás parámetros (memoria, interfaces, almacenamiento), hasta que cada panel esté marcado de la siguiente forma:



Para que Grafana pueda enviar correos, debemos configurar algunas cosas dentro del archivo grafana.ini, relacionadas al protocolo SMTP (Simple Mail Transfer Protocol)

SMTP habilitado: enabled = true permite el envío de correos.

Servidor y puerto: host = smtp.gmail.com:587 usa Gmail como servidor SMTP.

Usuario y contraseña: Se configura el correo de origen (user =) y la contraseña

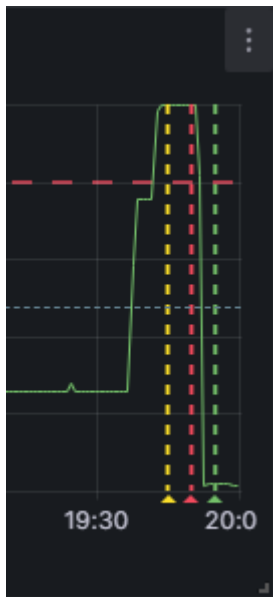
Remitente: from_address =

y from_name = Grafana Alerts especifican la dirección y nombre del remitente.

Y también, debemos configurar en el apartado de Alertas dentro del tablero de Grafana un punto de contacto, que en este caso son los correos a los que se van a mandar las diferentes alertas. Es decir: la configuración de SMTP del archivo grafana.ini es cómo Grafana va a mandar los correos, y el apartado de Contact Points es a qué correos va a mandar las alertas.

Prueba con el CPU

El comando `yes > /dev/null &` genera una carga constante en el procesador al crear un flujo infinito de datos, pero rediriéndolos a `/dev/null` para que no se muestren. Al ejecutarlo en segundo plano con `&`, puede estresar el procesador al 80% o más, especialmente si se usa varias veces.



Vemos en la gráfica de Grafana que efectivamente el uso del procesador supera el threshold.

 If there are problems with how this message is displayed, click here to view it in a web browser.
Click here to download pictures. To help protect your privacy, Outlook prevented automatic download of some pictures in this message.

Precaución: Este correo se originó desde fuera de la Institución. No haga clic en enlaces ni abra archivos adjuntos, a menos es seguro.

 Right-click or tap and hold here to download pictures. To help protect your privacy, Outlook prevented automatic download of some pictures in this message.

 **CPU > CPU**

An alert was triggered on Server 1

© 2025 Grafana Labs. Sent by [Grafana v10.2.6](#).

Debug: Plugin de Zabbix y SELinux

```

root@172-15-0-20:/home/adrian# sudo semodule -r old_denials 2>/dev/null
root@172-15-0-20:/home/adrian# sudo systemctl restart grafana-server
root@172-15-0-20:/home/adrian# sudo ausearch -m avc -ts recent | audit2allow -M grafana_zabbix
***** IMPORTANT *****
To make this policy package active, execute:

semodule -i grafana_zabbix.pp

root@172-15-0-20:/home/adrian# sudo semodule -i grafana_zabbix.pp
root@172-15-0-20:/home/adrian# sudo systemctl restart grafana-server
root@172-15-0-20:/home/adrian# getenforce
Enforcing
root@172-15-0-20:/home/adrian# vim /etc/grafana/grafana.ini
root@172-15-0-20:/home/adrian# |

```

sudo semodule -r old_denials: remueve módulos previos relacionados a denegaciones antiguas (limpieza).

sudo ausearch -m avc -ts recent | audit2allow -M grafana_zabbix: busca los últimos bloqueos (AVC) y genera un módulo de política SELinux llamado grafana_zabbix que permita esas acciones.

sudo semodule -i grafana_zabbix.pp: instala el módulo generado.

sudo systemctl restart grafana-server: reinicia el servicio de Grafana para aplicar cambios.

getenforce: muestra el modo actual de SELinux (por ejemplo, "Enforcing").

vim /etc/grafana/grafana.ini: se edita el archivo de configuración para permitir el plugin no firmado.

Script de backups

Este es el script en server2 (que ejecuta el de server 1 y hace sus backups)

```
#!/bin/bash
Date=$(date +%Y-%m-%d)
echo "Beginning compression and backup of /tmp/ folder on $Date"

#Preparing the files
mkdir -p /tmp/backups

cp /var/log/grafana/grafana.log /tmp/backups/grafana.log.bak
cp /var/log/zabbixsrv/zabbix_server.log /tmp/backups/zabbix_server.log.bak

FILE="/tmp/backups/Server2_logs-$Date.tar.bz2"
tar -cvjf "$FILE" /tmp/backups
if [ $? -ne 2 ]
then
    echo "Backup created successfully!"
    echo "Backup from $Date successfully created on /backups" >> /var/log/backup.log
    echo "Moving files over to Server 1"
    scp -P 2222 -i /home/itadmin/.ssh/id_ed25519 "$FILE" itadmin@172.15.0.10:/home/itadmin/backups/
    rm -f /tmp/backups/grafana.log.bak /tmp/backups/zabbix_server.log.bak "$FILE"
    echo "Executing script for backups in Server 1"
    ssh -t -p 2222 -i /home/itadmin/.ssh/id_ed25519 itadmin@172.15.0.10 'sudo /home/itadmin/backup_script.sh'
    PASSWORD="Apofis16"
    ssh -t -p 2222 -i /home/itadmin/.ssh/id_ed25519 itadmin@172.15.0.10 "echo '$PASSWORD' | sudo -S /home/itadmin/backup_script.sh"
    > /var/log/zabbixsrv/zabbix_server.log
    > /var/log/zabbixsrv/zabbix_server.log
else
    echo "Done"
else
    echo "Whoooooops! Something went wrong."
    echo "Error: Backup from $Date could not be created on /backups" >> /var/log/backup.log
fi
```

Primero, se copian los logs de Zabbix y Grafana a la carpeta */tmp/backups*. Después, se comprimen y dicho archivo se envía mediante SCP a server 1. Una vez hecho esto, se ejecuta remotamente un script mediante SSH, que está en Server 1 (más adelante explicaremos qué hace). Y finalmente, se vacían los logs originales de Zabbix y Grafana.

```
#!/bin/bash
Date=$(date +%Y-%m-%d)

#Preparing the files
LOG_MESSAGES="/var/log/messages"
LOG_HTTPD_ACCESS="/var/log/httpd/access_log"
LOG_WORDPRESS_ACCESS="/var/log/httpd/wordpress_access.log"
LOG_WORDPRESS_ERROR="/var/log/httpd/wordpress_error.log"

mkdir -p /tmp/backups

cp "$LOG_MESSAGES" /tmp/backups/messages.bak
cp "$LOG_HTTPD_ACCESS" /tmp/backups/httpd_access.bak
cp "$LOG_WORDPRESS_ACCESS" /tmp/backups/wordpress_access.bak
cp "$LOG_WORDPRESS_ERROR" /tmp/backups/wordpress_error.bak

FILE="/tmp/backups/Server1_logs-$Date.tar.bz2"
tar -cvjf "$FILE" /home/itadmin/backups
mv /tmp/backups/Server1_logs-2025-05-11.tar.bz2 /home/itadmin/backups/
|
if [ $? -eq 0 ]
then
    echo "Backup created successfully!"
    echo "Backup from $Date successfully created" >> /var/log/backup.log
else
    echo "Whoooooops! Something went wrong."
    echo "Error: Backup from $Date could not be created on /backups" >> /var/log/backup.log
fi
```

Este script se ejecuta en Server 1 y funciona de forma muy similar: se copian varios logs, se comprimen, y el archivo comprimido se mueve a la carpeta backups localizada en el directorio home del usuario *itadmin*, de tal manera que queda almacenado junto con el archivo con los logs de Server 2.

Como resultado, ambos backups están comprimidos y almacenados

```
itadmin@172-15-0-10:~$ ls backups
Server1_logs-2025-05-11.tar.bz2  Server2_logs-2025-05-11.tar.bz2
itadmin@172-15-0-10:~$ |
```

Como vemos que funciona y los backups se crean correctamente, lo volvemos un cronjob:

```
root@172-15-0-20:~# crontab -l
0 0 * * * /root/backup.sh
root@172-15-0-20:~# |
```

Hardening Server 2

```
root@172-15-0-20:/home/adrian# firewall-cmd --zone=work --list-all
work
target: default
ingress-priority: 0
egress-priority: 0
icmp-block-inversion: no
interfaces:
sources:
services: dhcpv6-client https mdns smtp snmp ssh
ports: 161/udp 2222/tcp
protocols:
forward: yes
masquerade: no
forward-ports:
source-ports:
icmp-blocks:
rich rules:
root@172-15-0-20:/home/adrian# |
```

En el firewall, nos aseguramos de abrir los puertos y servicios que están en uso (SNMP, SMTP, SSH puerto 22, HTTPS).

Ahora, para SELinux, primero creamos algunos usuarios de prueba, cada uno será asignado a un grupo distinto.


```

root@172-15-0-20:~# sudo useradd -G users norm_user2
root@172-15-0-20:~# sudo useradd -G guests guestuser2
root@172-15-0-20:~# sudo useradd -G admin itadmin
useradd: group 'admin' does not exist
root@172-15-0-20:~# sudo usermod -aG admins itadmin
root@172-15-0-20:~#

```

Verificamos que los usuarios pertenezcan en sus grupos correctos

```

root@172-15-0-20:/home/itadmin# groups itadmin
itadmin : itadmin wheel admins
root@172-15-0-20:/home/itadmin# groups norm_user2
norm_user2 : norm_user2 users
root@172-15-0-20:/home/itadmin# groups guestuser2
guestuser2 : guestuser2 guests
root@172-15-0-20:/home/itadmin# |

```

Y ahora, configuramos los diversos contextos de SELinux para cada uno de los usuarios, junto con el comando para ajustar el perfil de default, que será “guest_r”

sudo semanage user -a -R "sysadm_r" -P user admin_u: agregamos el rol sysadm_r al tipo de usuario admin_u

sudo semanage login -a -s admin_u itadmin: aplicamos el usuario admin_u al usuario de Linux itadmin

sudo semanage login -a -s user_u norm_user2: aplicamos el usuario user_u al usuario de Linux norm_user2

semanage login -m -s guest_u __default__: hacemos que el contexto predeterminado sea guest.

Y finalmente, para denegar el acceso por SSH al grupo guests, añadimos esta línea en */etc/ssh/sshd_config*.

DenyGroups guests

```

PS C:\Users\adria\Documents\Escuela\Materias\Servicios seguros> ssh itadmin@172.15.0.20 -p 2222 -i .\Server2
Last login: Sun May 11 15:35:39 2025
itadmin@172-15-0-20:~$ exit
logout
Connection to 172.15.0.20 closed.
PS C:\Users\adria\Documents\Escuela\Materias\Servicios seguros> ssh norm_user2@172.15.0.20 -p 2222 -i .\Server2
norm_user2@172-15-0-20:~$ exit
logout
Connection to 172.15.0.20 closed.
PS C:\Users\adria\Documents\Escuela\Materias\Servicios seguros> ssh guestuser2@172.15.0.20 -p 2222 -i .\Server2
guestuser2@172.15.0.20: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
PS C:\Users\adria\Documents\Escuela\Materias\Servicios seguros> |

```

Después de estos ajustes, se puede comprobar que los usuarios pertenecientes al grupo guests no pueden acceder por SSH, mientras que los demás sí. Como nota, para que el usuario itadmin pueda tener acceso, es necesario cambiar un valor en SELinux con el siguiente comando, ya que dicho usuario corre bajo el contexto de sysadmin.

sudo setsebool ssh_sysadm_login 1.

Observamos que los contextos están correctamente aplicados.

```
norm_user2@172-15-0-20:~$ id -Z
user_u:user_r:user_t:s0
norm_user2@172-15-0-20:~$ |
```

```
itadmin@172-15-0-20:~$ id -Z
admin_u:sysadm_r:sysadm_t:s0
itadmin@172-15-0-20:~$ |
```

Conclusión

Este proyecto ha sido fundamental para consolidar diversos aspectos de la administración de servidores Linux en un entorno controlado, con un enfoque particular en la configuración y despliegue de dos servidores que no solo cumplen con funciones específicas, sino que también implementan prácticas clave de seguridad y automatización.

Uno de los logros más relevantes fue la configuración del primer servidor con un stack LAMP, implementando HTTPS con certificados autofirmados y la resolución de nombres mediante DNS, que nos permite en cierto grado un entorno seguro, además del segundo servidor centrado en SNMP, ZABBIX y Grafana. Además de ellos y justamente hardening del sistema, aplicamos firewalld para segmentar el traffic y selinux siempre en modo enforcement, así mismo creamos por ejemplo los certificados autofirmado para loguear en servicios como wordpress, muy muy importante y esas prácticas que sin duda en su ausencia serian altamente explotables

En resumen, considero que el proyecto integra de manera correcta el uso de los conocimientos aprendidos, conjunta todo y es un grado de dificultad adquirido, el hecho de hacer convivir las aplicaciones y las dos máquinas requieren un correcto deployment de las aplicaciones y no solo eso sino que en lo personal destaco el uso del mismo sistema operativo, saber moverse en el entorno para mi es la mitad de la práctica.

Referencias

1. Grafana Labs. (n.d.). *Zabbix plugin for Grafana*. (grafana.com)
2. Zabbix. (n.d.). *Download and install Zabbix 7.2 for Red Hat Enterprise Linux 9*. (zabbix.com)
3. Codex, A. C. (2024, January 27). *Configuring System Logging with Rsyslog on Fedora 38*. Reintech. (reintech.io)

Hola:), me gustaría compartir este proyecto para mostrar un poco todo lo que aprendí durante el semestre además me interesa además compartir documentación relacionada a este tema, es de la materia de servicios seguros de internet. Donde trabajamos con fedora la cual es un distro de linux o sea una versión de linux.

Proyecto final de semestre: Despliegue de dos servidores funcionales en Fedora Linux

El primero fue un servidor web con stack LAMP (Linux, Apache, MySQL, PHP) y HTTPS usando certificados autofirmados, además de configurar el servicio de DNS con Bind9 y centralizar logs mediante rsyslog.

Segundo servidor

Para este segundo, por su parte, el segundo servidor será responsable de la supervisión y monitoreo continuo del sistema, utilizando SNMP, Zabbix y Grafana, además de ejecutar tareas programadas para respaldos y gestión de logs.

Hardening

Ambos servidores deben cumplir con políticas de seguridad, entre ellas: firewalld y la aplicación de políticas de SELinux activa (Enforcing), y la segmentación de usuarios en grupos con distintos niveles de privilegios.

Asimismo, se aplicarán prácticas de hardening como la restricción del acceso SSH al usuario root, el uso exclusivo de autenticación por llaves, la modificación del puerto por defecto y la prohibición del login por contraseña.

Lo personalmente aprendido

Con esto se cerró el curso, y me gustaría expresar lo mucho que aprendí. En gran parte, esto se debió al profesor Daniel.

Se me hace increíble que pude manejar mejor Linux y comprender mucho más su estructura. Aprender a buscar documentación sin atajos.

Siento que a veces hay una gran barrera al tratar de empezar en la ciberseguridad ofensiva, la cual es carecer de algunos fundamentos técnicos. Definitivamente, durante este tiempo he entendido más, a nivel de sistema operativo, la funcionalidad de Fedora y en general de los servicios Linux.

Por no mencionar todas las tecnologías usadas, el troubleshooting y sobre todo la persistencia y disciplina, porque aunque se presentan dificultades, uno trata y tiene que ver la manera de encontrar una solución. Definitivamente, una de mis clases favoritas, y gran parte de ello es crédito del profesor.

Tecnologías: Fedora Linux, Apache, MySQL, PHP, Bind9, Rsyslog, SSL, SELinux, FirewallD.

.